POKEDEX

ID:209094439

[Report]

Table of Contents

Pokedex ***2***

Guide: **2**

Pokémon Struct: **2**

Operations: **3**

PokemonList: ***4***

PokemonNode: **4**

New Node: **4**

PlayerList: ***5***

Traversing: **5**

Process: **6**

Summary ***7***

Sources ***8***

Pokedex

A Pokedex is diary in which Pokémon's and Player's catalogue are store, basically implementing a series of code with the ability to link together and working perfectly, we want to work through the steps for functionally, so it's better to focus on one step at the time meaning adding one feature at the time to avoid confusion. Starting off with virtual code using C++ and creating a new folder containing files for each for each step in the memory. Then push my folder on Gitlab at the end.

Guide: This reading is a guide on how the players catch their Pokémon using C Linked Lists, creating an easier way to store both PokemonList and PlayerList in the data structure, with the access of pointers, node, parameters, and variable to create a link between them. We will go over the certain strategies when to add or remove pointers, connect pointers & nodes, removing nodes and orderly go through steps being used in the process, our focus in structure is the ability to search, find and capture for a player without any creating any blank space in the memory, and if so, how we approach this type of situation without making it worse? The Pokedex is viewed as a card shuffle for Pokémon to players with the ability to display the Pokémon and player details in the pokedex with name, all the Pokémon owned in the list, then listing all Pokémon and player in the pokedex.

```
#include <string.h>
#include <stdlib.h>
#include <stdio.h>
```

Pokémon Struct: Understanding the core Data Structure, the importance of using Basic Operation and to avoid situations such as overflow meaning adding a lot to an integer causing it to go from a positive to negative side, underflows the opposite of Overflow with the lack of storage in the data and direct memory access are very important in the Pokémon Struct "sometimes we access memory directly (much of the rest of this module is about this), we need to how the memory is being used to understand its contents"- Programming 03 (2022). How do we implement the structure using C Memory, this could be used for static data and heap, we want to access the data of the structure directly from the pokedex with the help of Direct memory access which give us permission to get a hand in memory directly , Data Structure enable us to add any arrays or variables into the struct, we can use keyword as struct, structure tag and structure variable.

```
//struct store info of a Pokémon
struct PokemonNode {
    char name[15];
```

```
    char ability[15];
    char type[15];
    struct PokemonNode *next;
    struct PokemonNode *prev;

};
```

```
//struct to store in the Pokedex
struct Pokedex {
    struct PokemonNode *headPokemon;
    struct PlayerNode *headPlayer;
};
```

Operations: Moving into the typedef which let the structure know what type we are using, then we want to access the struct members using member access operator(.), where we can use syntax such as (.& ->) to define members of class or struct which also help us finds the value, PokemonList & PlayerList are structure in the data structure which means using typedef struct twice to identify the amount being added in the data and any pointers directing to a node for both PokemonList & PlayerList, setting Pokémon as type of struct and applying integer values to store the numerical value in the process and using char counting the spacing each line for the Pokémon such as Pokedex, name, type, ability and pokemon_id.

```
// In order to create a new Player node  we need to apply a Function
struct PlayerNode *createPokemonNode(char name[]) {
    struct PlayerNode *NWnode = (struct PlayerNode *)malloc(sizeof(struct
PlayerNode));
    strcpy(NWnode->name, name);
    NWnode->PokemonCount = 0;
    NWnode->next = NULL;
    NWnode->prev = NULL;
    return NWnode;
}
```

PokemonList:

PokemonNode: This process will require a few steps for the PokemonList to work. Firstly, to create a node that links the PokemonNode and the NewPokemonNode, we want to set the head to null for that we will require stdlib.h for malloc "Set aside space in memory for a node structure. If this fails new_node will be set to NULL"- Programming 03 (2022), which set a large space in the memory enough to handle one of the structs and using sizeof in case of failure the new node will go to NULL, adding an IF statement afterward to ensure the node before using it, assign the data values to the next pointer to NULL.

```
// To add new Pokémon into list  we need to apply a Function

void addPokemonToList(struct PokemonNode *NWnode){
    if (pokedex.headPokemon == NULL){
        pokedex.headPokemon = NWnode;
    } else {
        struct PokemonNode *current = pokedex.headPokemon;
        while (current->next != NULL) {

        }
        current-> next = NWnode;
        NWnode->prev = current;
    }
}
```

New Node: Secondly, we want to create another note and add the Pokémon to the list in which the node and the new node is assign an integer value then Returning the new node. Thirdly, search the Pokémon in the pokedex and returning it to the pointer, we want to create a library which stores the Pokémon as the type and storing Pokémon characteristic value such as the name, type, and ability in pokedex, using char setting amount of space on the line [15], then returning it to NULL. Adding Pokemon_ID and using strcpy "When you use strcpy (), the size of the destination string should be large enough to store the copied string. Otherwise, it may result in undefined behavior"- Programiz (2022), which means to add the new node value in the library for each characteristic then a next function returning it to NULL to create the cycle into the pokedex.

```
// To search for a Pokémon using name we need to apply a Function
struct PokemonNode *searchPokemonByName(char name[]){
    struct PokemonNode *current = pokedex.headPokemon;
    while (current != NULL){
```

```
        if (strcmp(current->name, name, name)) == 0) {
            return current;
        }
        current = current->next;
    }
    return NULL;
```

PlayerList:

Traversing: Apply the same process for the players but traversing the list, "To traverse a list, that is to iterate through the list, we start at the `head` and keep moving through the `nodes` by using the `next` pointer until `next == NULL` and we have reached the last node "- Programming 03 meaning going through the list starting at the head using next to go through all the nodes until it says next == NULL indicating the last node and applying curr (current), Next, and prev (previous) to move the node from the current position to the next position .

```
//struct store info of Player
struct PlayerNode {
    char name[15];
    int PlayersNumber;
    int PokemonCount;
    struct PokemonCount *PlayerPokemon[100];
    struct PlayerNode *next;
    struct PlayerNode *prev;
};
```

```
//To add a new Player to the List we need to apply a Function
void addPlayerToList(struct PlayerNode *NWnode){
    if (pokedex.headPlayer == NULL) {
        pokedex.headPlayer == NWnode;
    } else {
        struct PlayerNode *current = pokedex.headPlayer;
        while (current-> next != NULL ) {
            current = current-> next;
        }
        current->next = NWnode;
        NWnode->prev = current;

    }
}
```

Process: Starting the Process by coping the head, using a while statement to see if it's the last node and changing the pointer of the current temp to the next temp then afterward print out the values in the order, we assign it to. Moving to the next step of adding a Pokémon to a player, we want to set a start and integer value in the void of the Pokémon to the player linking it to store other data.

```
//To add a new Player to the List we need to apply a Function
void addPlayerToList(struct PlayerNode *NWnode){
    if (pokedex.headPlayer == NULL) {
        pokedex.headPlayer == NWnode;
    } else {
        struct PlayerNode *current = pokedex.headPlayer;
        while (current-> next != NULL ) {
            current = current-> next;
        }
        current->next = NWnode;
        NWnode->prev = current;

    }
}
```

Using Main to start an empty list and storing the Pokémon entries from head to eight, then taking the pointer to the head using alternative, adding a start to note and integer value to the alternative. Another temp to the next returning null in case the next null is the last entry of the list, or it's been broken. Afterward in order to store pointer to the node in the pokemonList for the Pokémon that player has captured, we want to using insertNode to store the head and any numeric value, then traverse it until you find the note of your choosing to insert meaning the note before set to null, creating a possibility to insert a second node meaning the note after set to null and applying the same method for the pointer to the node that are before and the one after. To properly connect a new node in the correct place to insert it to the node after we want to direct the current note to the next and the temp to the one after, otherwise it will create an error. Lastly, we want to remove the node using deleteNode to connect it safely to the node and to the node after that then removing it free up the memory, then use the head to pass a pointer setting it to a two single pointers , once we find the node we remove the temp pointer keeping it a record of the past node, then adding free(delete) to free up the memory in the code.

```c
// To create a new Player node we need to apply a Function
struct PlayerNode *createPokemonNode(char name[]) {
    struct PlayerNode *NWnode = (struct PlayerNode *)malloc(sizeof(struct
Playernode));
    strcpy(NWnode->name, name);
    NWnode->PokemonCount = 0;
    NWnode->next = NULL;
    NWnode->prev = NULL;
    return NWnode;
```

Summary

To conclude, the ideal pokedex should include a player, with its number and a Pokémon assign. Our pokedex should create a connect from the pokedex to players and the Pokémon in a form until NULL and could design a battle between players in which they are in alphabetical and numeric order, players could be knocked out with a winner at the end makes it more fun. Afterward, we want to run some testing of the pokedex (test_pokedex.C), it's important it works and can used by anybody, using unit testing, which is one of the easiest methods of test, it automated each function to testing pleasantly. Using VSCode at the end to push my file into Gitlab, first we want to download some of the extension need to clone a repository once it's done It's easier to get the project onto GitLab or use terminal git codes to send the file onto the GitLab project "The code you cloned from the original source will now be uploaded to your repository. Note any changes you have made will not be uploaded. To upload your changes follow the Upload Changes section below"-Programming 03 (2022).

.

```c
//To add a new Player to the List we need to apply a Function
void addPlayerToList (struct PlayerNode *NWnode){
    if (pokedex.headPlayer == NULL) {
        pokedex.headPlayer == NWnode;
    } else {
        struct PlayerNode *current = pokedex.headPlayer;
        while (current-> next != NULL ) {
            current = current-> next;
        }
        current->next = NWnode;
        NWnode->prev = current;

    }
}
```

Sources

COM5001-Programming 03. (2022). Reference – Visual Studio Code. [online] Available at: https://ysjprog03.netlify.app/docs/home/ . [Accessed January 2, 2022].

COM5001-Programming 03. (2022). C Programming. [online] Available at: https://ysjprog03.netlify.app/docs/home/ . [Accessed January 2, 2022].

COM5001-Programming 03. (2022). Basic Operations - Summary. [online] Available at: https://ysjprog03.netlify.app/docs/home/ . [Accessed January 3, 2022].

COM5001-Programming 03. (2022). Memory – Physical Memory. [online] Available at: https://ysjprog03.netlify.app/docs/home/. [Accessed January 5, 2022].

COM5001-Programming 03. (2022). C Struct - C Data Structures. [online] Available at: https://ysjprog03.netlify.app/docs/home/. [Accessed January 10, 2022].

COM5001-Programming 03. (2022). C Data Structures – C Linked Lists. [online] Available at: https://ysjprog03.netlify.app/docs/home/ . [Accessed January 12, 2022].

COM5001-Programming 03. (2022). Visual Studio Code. [online] Available at: https://ysjprog03.netlify.app/docs/home/ . [Accessed January 12, 2022].

COM5001-Programming 03. (2022). VSC Cloning. [online] Available at: https://ysjprog03.netlify.app/docs/home/ . [Accessed January 12, 2022].

COM5001-Programming 03. (2022). Git. [online] Available at: https://ysjprog03.netlify.app/docs/home/ . [Accessed January 12, 2022].

COM5001-Programming 03. (2022). VSC GitLab. [online] Available at: https://ysjprog03.netlify.app/docs/home/ . [Accessed January 12, 2022].

COM5001-Programming 03. (2022). Terminal; GitLab. [online] Available at: https://ysjprog03.netlify.app/docs/home/ . [Accessed January 12, 2022].

Programiz. (2022). C strcpy (). [online] Available at: https://www.programiz.com/c-programming/library-function/string.h/strcpy . [Accessed January 14, 2022].

GitLab Project Link : https://git.ysjcs.net:8888/paolo.tokam/pokedex_01.git